

# CLAUDE.md - Containers Module

## INHERITED FROM the Helix Constitution

This module is governed by the Helix Constitution. All rules in the constitutionâ€™s CLAUDE.md and the Constitution.md it references apply unconditionally. Locate the constitution from any nested depth via its `find_constitution.sh` helper â€” do NOT hardcode a path (this module stays fully decoupled and project-agnostic per Â§11.4.28).

Canonical reference: <https://github.com/HelixDevelopment/HelixConstitution>

## Acceptance demo for this module

```
# Real orchestration flow: boot a container via pkg/boot.BootManager and
# verify its health check passes against a live runtime (Docker/Podman).
cd Containers && GOMAXPROCS=2 nice -n 19 go test -tags=integration -count=
```

Expect: PASS; exercises `runtime.AutoDetect`, `boot.BootManager.BootAll`, and `health.HealthChecker` against a real container runtime per this moduleâ€™s README.md Quick Start. Requires a rootless Docker/Podman runtime available to the current user.

## Overview

`digital.vasic.containers` is a generic, reusable Go module for container orchestration, health checking, lifecycle management, and service discovery. It supports Docker, Podman, and Kubernetes runtimes.

**Module:** `digital.vasic.containers` (Go 1.24+)

## Build & Test

```
go build ./...
go test ./... -count=1 -race
go test ./... -short           # Unit tests only
go test -tags=integration ./... # Integration tests (requires Docker)
go test -bench=. ./tests/benchmark/
```

## Code Style

- Standard Go conventions, `gofmt` formatting
- Imports grouped: `stdlib`, third-party, internal (blank line separated)
- Line length â‰¤ 100 chars
- Naming: camelCase private, PascalCase exported, acronyms all-caps
- Errors: always check, wrap with `fmt.Errorf("...: %w", err)`
- Tests: table-driven, `testify`, naming `Test<Struct>_<Method>_<Scenario>`

# Package Structure

| Package          | Purpose                                                   |
|------------------|-----------------------------------------------------------|
| pkg/runtime      | Container runtime abstraction (Docker/Podman/K8s)         |
| pkg/compose      | Docker Compose orchestration                              |
| pkg/health       | Health checking (TCP/HTTP/gRPC/Custom)                    |
| pkg/endpoint     | Service endpoint configuration                            |
| pkg/lifecycle    | Advanced lifecycle (lazy boot, idle shutdown, semaphores) |
| pkg/monitor      | Resource monitoring (CPU/memory/disk), cluster snapshots  |
| pkg/event        | Event bus for lifecycle hooks (20 event types)            |
| pkg/discovery    | Service discovery (TCP/DNS)                               |
| pkg/logging      | Logging abstraction (bring your own)                      |
| pkg/metrics      | Prometheus-compatible metrics                             |
| pkg/boot         | High-level BootManager composing everything               |
| pkg/remote       | Remote host management, SSH executor, connection pooling  |
| pkg/scheduler    | Resource-aware container scheduling (5 strategies)        |
| pkg/network      | SSH tunnel management, port allocation, overlay networks  |
| pkg/volume       | Remote volume management (SSHFS/NFS/rsync)                |
| pkg/envconfig    | Environment-variable-based configuration for remote hosts |
| pkg/distribution | Distribution orchestrator: schedule, deploy, failover     |

## Key Interfaces

- `runtime.ContainerRuntime` â€” Container operations (local and remote via `RemoteRuntime`)
- `compose.ComposeOrchestrator` â€” Compose file operations (local and remote)
- `health.HealthChecker` â€” Health check dispatch
- `lifecycle.LifecycleManager` â€” Service lifecycle with lazy boot
- `monitor.ResourceMonitor` â€” System/container resource monitoring
- `event.EventBus` â€” Publish/subscribe for lifecycle events (20 event types)
- `discovery.Discoverer` â€” Service discovery
- `logging.Logger` â€” Logging abstraction
- `metrics.MetricsCollector` â€” Metrics collection
- `remote.RemoteExecutor` â€” SSH command execution on remote hosts
- `remote.HostManager` â€” Remote host registry and resource probing
- `scheduler.Scheduler` â€” Resource-aware container placement (5 strategies)
- `network.TunnelManager` â€” SSH tunnel creation/management
- `volume.VolumeManager` â€” Remote volume mounting (SSHFS/NFS/rsync)
- `distribution.Distributor` â€” Unified distribution orchestrator

# Design Patterns

- **Strategy:** ContainerRuntime (Docker/Podman/K8s), HealthChecker (TCP/HTTP/gRPC), Scheduler (5 strategies)
- **Observer:** EventBus for lifecycle events (20 event types)
- **Factory:** runtime.AutoDetect(), health.NewDefaultChecker()
- **Builder:** endpoint.NewEndpoint().WithHost().WithPort().Build()
- **Decorator:** RetryPolicy wraps HealthChecker, RemoteRuntime wraps ContainerRuntime
- **Functional Options:** boot.WithRuntime(), distribution.WithScheduler(), etc.
- **Proxy:** RemoteRuntime routes ContainerRuntime calls via SSH
- **Facade:** Distributor composes scheduler + remote + network + volume

## Composition: how the pieces combine

The adapter layer that the consuming project uses (internal/adapters/containers/adapter.go) wires the module together as follows:

the consuming project BootManager â†’ Adapter.BootAll(endpoints)

```
â”,
â”€â”€â”€â”€ ContainerRuntime (auto-detected: Docker / Podman / con
â”€â”€â”€â”€ ComposeOrchestrator (compose file parse + up/down, loc
â”€â”€â”€â”€ HealthChecker (TCP / HTTP / gRPC, with retry)
â”,
â”€â”€â”€â”€ (if CONTAINERS_REMOTE_ENABLED=true)
DefaultDistributor
â”,
â”€â”€â”€â”€ Scheduler (chooses host per container: resource_av
â”€â”€â”€â”€ RemoteRuntime = proxy(ContainerRuntime) over SSHExe
â”€â”€â”€â”€ TunnelManager (SSH port forwarding for cross-host
â”€â”€â”€â”€ VolumeManager (SSHFS / NFS / rsync)
```

Distributor receives a batch of container requirements, asks Scheduler which host each should land on (local or a named remote), then either calls the local runtime directly or wraps it in RemoteRuntime for SSH execution.

## Mandatory Container Orchestration Flow (inline)

This is what the consuming projectâ€™s root CLAUDE.md Hard Stop #2 refers to. The flow is:

1. **Build:** make build â†’ ./bin/helixagent.
2. **Env load:** the consuming project reads Containers/.env via envconfig.LoadFromFile():
  - CONTAINERS\_REMOTE\_ENABLED (bool)
  - CONTAINERS\_REMOTE\_HOST\_N\_\* (N = 1..100; loader stops at the first absent \_NAME)
  - SSH pool, timeouts, scheduler strategy
3. **Adapter init** (internal/adapters/containers/adapter.go, NewAdapterFromConfig):
  - runtime.AutoDetect() picks the local container runtime.

- If remote enabled: build `SSHExecutor` with `ControlMaster` pooling; create `HostManager`; register all remote hosts; create `Scheduler` (default strategy: `resource_aware`); construct `DefaultDistributor`.
- 4. **Service boot** (`BootManager.BootAll`):
  - Register endpoints (name, compose file, health check, remote flag).
  - For each endpoint with a compose file: `Adapter.ComposeUp()` â†’ local compose or remote compose-via-SSH.
  - Remote compose: `SCP` compose file + build contexts to host, `docker compose -f <file> up -d`.
  - Health checker probes each service (TCP / HTTP). Required services failing = boot failure.
- 5. **Container distribution** (optional, on explicit request):
  - Caller supplies `[]ContainerRequirements` (name, image, CPU / mem / GPU, labels).
  - `Distributor.Distribute()` â†’ `Scheduler.ScheduleBatch()` â†’ probes hosts â†’ assigns each container to the best host.
  - For each container: `SSH docker run -d` on assigned host, create tunnels, mount volumes.
  - Returns `DistributionSummary` (local count, remote count, failures).
- 6. **Health & monitoring (continuous)**: periodic `HealthChecker.CheckAll()` + `HostManager.ProbeAll()` for re-balancing inputs.
- 7. **Shutdown**: `Adapter.Shutdown()` â†’ `Distributor.Undistribute()` â†’ close SSH tunnels, unmount volumes, `ComposeDown()` on each compose file.

**The correct workflow is** `make build` â†’ `./bin/helixagent`. Never run `docker compose up` / `podman-compose up` / `make test-infra-start` manually â€” they bypass this flow and produce the â€œworks on my machineâ€” class of incident that `CONST-030` exists to prevent.

## Remote Distribution

**Env-var registration** (`pkg/envconfig/parser.go`):

`CONTAINERS_REMOTE_HOST_N_*` entries, `N = 1..100`. The loader iterates until a missing `_NAME` is hit.

```
CONTAINERS_REMOTE_HOST_1_NAME=gpu-server-1
CONTAINERS_REMOTE_HOST_1_ADDRESS=192.168.1.100
CONTAINERS_REMOTE_HOST_1_PORT=22
CONTAINERS_REMOTE_HOST_1_USER=deploy
CONTAINERS_REMOTE_HOST_1_KEY=~/.ssh/id_rsa
CONTAINERS_REMOTE_HOST_1_RUNTIME=docker
CONTAINERS_REMOTE_HOST_1_LABELS=gpu=true,arch=amd64
```

Adding a host = append six env vars. No code change, `N` scales freely (this is `CONST-031`).

**Deployment loop** (`pkg/distribution/distributor.go`): for each placement decision, if local â†’ `LocalRuntime.Start(image)`; else â†’ `SSH docker rm -f <name> 2>/dev/null || true` then `docker run -d --name <name> <image>`, then `TunnelManager.CreateTunnel()`, then `VolumeManager.Mount()`, then remote health check.

**SSH ControlMaster pooling** (`pkg/remote/connection_pool.go`): one socket per `(user@host:port)` in `/tmp/containers-ssh-ctrl/`. `Acquire()` creates the socket if missing and bumps a ref count; `Release()` decrements. Socket persists for

`ControlPersist` (default 5 min) after ref count hits zero â€” massive latency reduction for rapid successive calls.

**Scheduler strategies** (`pkg/scheduler/strategies.go`): `resource_aware` (default), `round_robin`, `affinity`, `spread`, `bin_pack`.

## Gotchas

1. **ControlMaster socket semantics:** the socket can outlive the last `Release()` by `ControlPersist`. If the network blips during that window, queued commands can hit a dead socket. Always `IsReachable()`-probe before assuming a host is live.
2. **CommandTimeout vs. KeepAlive:** `CONTAINERS_REMOTE_COMMAND_TIMEOUT` (default 1800s) bounds the outer SSH command. `ServerAliveInterval`—`ServerAliveCountMax = 30s / 10 = 5 min` heartbeat tolerance. Never set `CommandTimeout < KeepAliveTotal`, or long compose builds with multi-GB image pulls will appear to hang and then die.
3. **Context cancellation in ScheduleBatch:** host probes run synchronously. If `ctx` cancels mid-probe, Scheduler uses whatever snapshots it has â€” placements may be suboptimal rather than failing. Use a realistic deadline.
4. **Build-context skip:** `RemoteComposeUp` SCPs build contexts to the remote host *except* when the context path matches the project root (via `filepath.Clean` comparison). `build: { context: . }` pointing at the consuming projectâ€™s root is silently skipped so the whole 27 GB tree isnâ€™t shipped. This is intentional.
5. **Volume timing:** `VolumeManager` mounts volumes *after* container start. If a container needs the volume at bind-mount time (read-only config at entrypoint), it fails. Use retrying health checks or init containers that wait for the mount.
6. **No auto-failover:** a failed container is not moved to a backup host automatically. `Distribute()` is not idempotent; `Undistribute()` is. Call `Rebalance()` or the `Undistribute` â†’ `Distribute` pair to retry.

## Key files a developer touches

- `pkg/distribution/distributor.go` â€” placement + deployment orchestration.
- `pkg/scheduler/scheduler.go` + `strategies.go` â€” scheduling logic; add new strategies here.
- `pkg/remote/ssh_executor.go` â€” SSH execution, timeouts, streaming output.
- `pkg/remote/host_manager.go` â€” host registry; add host auto-discovery / state callbacks here.
- `pkg/envconfig/parser.go` â€” env-var loader; add new `CONTAINERS_REMOTE_*` variables here.
- `pkg/orchestrator/orchestrator.go` â€” multi-service boot ordering, rollback.
- The consuming project side: `internal/adapters/containers/adapters.go` â€” the single integration point.

## Integration Seams

- **Upstream:** none (this module is foundational).
- **Downstream (sibling modules):** `Challenges`, `HelixLLM`, `HelixQA`.
- **The consuming project consumers:** `internal/adapters/containers/adapters.go`, `internal/services/boot_manager.go`.

- **Hard external dependencies:** SSH client binaries, a container runtime on the local machine (Docker/Podman/etc.), SSH server + container runtime on each configured remote host, SSH network reachability.

## Commit Style

Conventional Commits: `feat(runtime): add Kubernetes support`